

Full length article

MIRDETECTOR: Applying malicious intent representation for enhanced APT anomaly detection

Hongmei Li^a, Tiantian Zhu^{a,*}, Jie Ying^a, Tieming Chen^a, Mingqi Lv^a, Jian-Ping Mei^a, Zhengqiu Weng^{b,*}, Lili Shi^c

^a Zhejiang University of Technology, China

^b Wenzhou University of Technology, China

^c Wenzhou Polytechnic, China

ARTICLE INFO

Keywords:

APT attack
Anomaly detection
Deep learning
Malicious intent representation
Real-time detection
Host security
Provenance-based intrusion detection

ABSTRACT

Advanced Persistent Threats (APTs) infiltrate target systems covertly, exhibiting behavior that is difficult to detect using conventional detection methods. Posing significant risks to enterprise security. Data provenance technology is widely used in attack detection to counter these threats. Among the different types of Provenance-based Intrusion Detection Systems (PIDSes), anomaly-based PIDSes are gaining increasing attention due to their ability to counter zero-day vulnerabilities without relying on attack knowledge. The detection mechanism of anomaly-based PIDSes is based on modeling the system's normal behavior patterns (structural/attribute features) to detect deviations in behavior. However, existing anomaly-based PIDSes are prone to a significant number of false positives due to benign data fluctuations, limiting their effectiveness against complex APT attacks. To address this, we propose MIRDETECTOR, a novel anomaly detection system for APT attacks. The core idea of MIRDETECTOR is that a node is considered malicious not only due to changes in its structural/attribute features but also because it exhibits a certain inclination toward malicious intent. Building on this idea, MIRDETECTOR models nodes from three dimensions: structural features, attribute features, and malicious intent representation. By employing lightweight models for training and detection, it effectively reduces the false positives and achieves efficient real-time detection. We have thoroughly evaluated MIRDETECTOR on several public datasets and compared it with state-of-the-art anomaly detection systems. The results demonstrate that MIRDETECTOR achieves excellent detection accuracy and recall. Compared to the baseline detection system, MIRDETECTOR has increased the node-level detection accuracy by up to 99% and the recall rate by up to 68%. This significantly mitigates the high false positives in traditional PIDSes that rely solely on structural/attribute features. MIRDetector demonstrates remarkable accuracy and efficiency in identifying complex threats. Its deployment will effectively mitigate the risks posed by APTs.

1. Introduction

In recent years, cyber security threats to large enterprises have grown increasingly severe, with APTs being a notable example (Navarro et al., 2018). These attacks, typically launched by skilled attackers with sophisticated cyber capabilities (Lockheed Martin, 2022), are marked by a high degree of organization and a clear target focus. They often exploit zero-day vulnerabilities and employ a “slow and low-key” approach to infiltration. The success of APT attacks can lead to significant economic damage and political risks (Alshamrani et al., 2019). Traditional methods of attack investigation (He et al., 2016; Murínová, 2015) frequently struggle to identify attacks due to a lack of concrete evidence. To counter these threats and precisely pinpoint attack incidents for investigation, the research community has introduced

data provenance technology (Inam et al., 2022). This technology maps the system's execution history through an audit provenance graph, providing rich contextual information. Analysts can leverage this data to conduct causal analyses of system activities, detect intrusions, trace interdependencies, and comprehend security incidents (Fang et al., 2021; Ying et al., 2024; Alsaheel et al., 2021).

Drawing from the provenance graph, numerous detection systems have been developed. These systems capitalize on the rich contextual information offered by the provenance graph to conduct host-based intrusion detection, known as **Provenance-Based Intrusion Detection Systems (PIDSes)**. Presently, PIDSes are primarily categorized into rule-based and anomaly-based PIDSes. Rule-based PIDSes, as seen in

* Corresponding authors.

E-mail addresses: ttzhu@zjut.edu.cn (T. Zhu), derisweng@wzut.edu.cn (Z. Weng).

<https://doi.org/10.1016/j.cose.2025.104588>

Received 3 February 2025; Received in revised form 8 May 2025; Accepted 28 June 2025

Available online 11 July 2025

0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

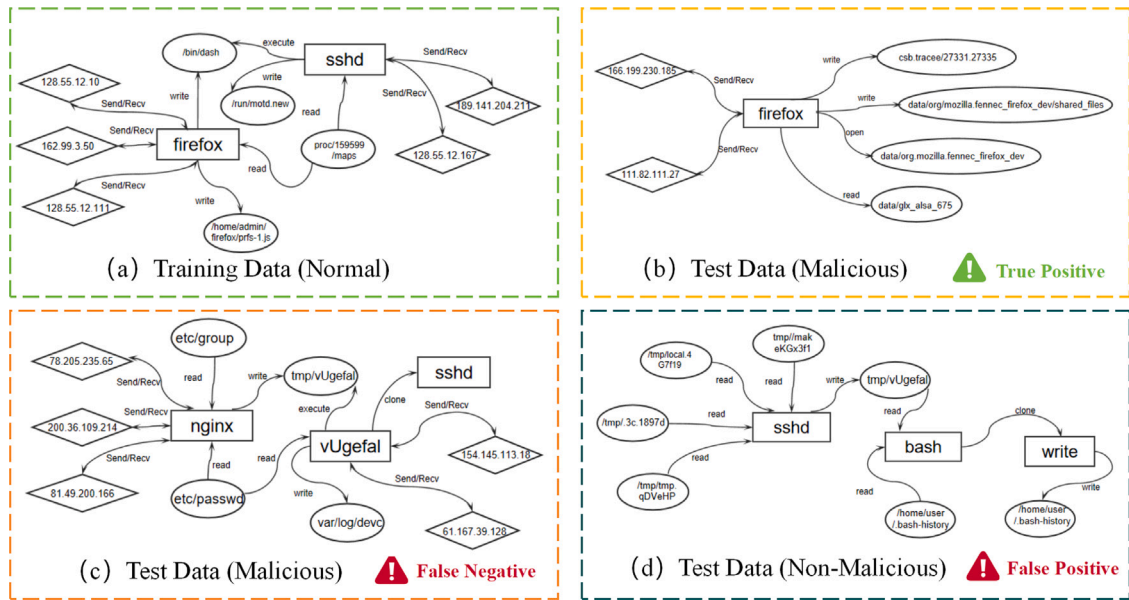


Fig. 1. The example diagrams illustrate the application of PIDSeS in different scenarios. Figure (a) is a benign graph extracted from DARPA's E3-THEIA dataset, which represents the data that PIDSeS learn during the training phase. Figures (b), (c), and (d) depict potential detection outcomes of PIDSeS when confronted with varying data during the detection phase. Specifically, Figure (b) presents an attack graph derived from DARPA's E3-ClearScope dataset, Figure (c) displays an attack graph from DARPA's E5-CADETS dataset, and Figure (d) shows a benign graph from DARPA's E3-THEIA dataset.

works such as Milajerdi et al. (2019b), Hassan et al. (2019a), Hossain et al. (2017a), Milajerdi et al. (2019a), Najafi et al. (2024) and Lv et al. (2024), operate by establishing manual policies. The core concept is to use threat intelligence to create rules, and any behavior that matches these predefined threat rules is flagged as malicious. Anomaly-based PIDSeS, which include studies like Han et al. (2020), Hassan et al. (2018), Han et al. (2017) and Liu et al. (2018), function by learning the patterns of benign behavior within a system. The underlying principle is that malicious behaviors typically deviate significantly from benign ones. Anomaly-based PIDSeS model the benign behaviors of a system and flag any behaviors that significantly deviate from this model as abnormal. While anomaly-based detection methods have been highly successful in detecting unknown zero-day attacks, we have observed that they are prone to generating a large number of false positives and are susceptible to evasion attacks, as discussed in Zügner et al. (2018) and Wang and Gong (2019).

To integrate provenance graphs into machine learning models, the majority of current detection systems transform *graphs/path/nodes* into a fixed-length vector representation, leveraging their surrounding structural information (Han et al., 2020, 2021; Cheng et al., 2023; Jia et al., 2024). The fundamental premise of these detection methods is that the behavioral characteristics of malicious *graphs/paths/nodes* are distinct from those of benign ones. From a modeling standpoint, this implies that the structural features of relevant malicious *graphs/paths/nodes* should also differ from those of benign ones. However, as is widely acknowledged, it is unfeasible to capture all normal behaviors (Pang et al., 2021). If a benign behavior is not learned, the detection system may trigger an alert, which accounts for the high incidence of false positives in traditional intrusion detection methods. Consequently, anomaly-based detection methods that solely model the structural features of the system's benign behavior are inadequate.

The core principle of anomaly-based detection methods is the inherent difference in structural characteristics between malicious and benign behavior. These methods encounter two main challenges: first, they are highly susceptible to the influence of unlearned benign behaviors; second, they are inevitably affected by evasive tactics employed in attacks, as recent research by Goyal et al. (2023) has demonstrated. Critiquing the traditional intrusion detection methods' insights leads us

to a critical question: What insight drives our detection method? To clarify this, we will use several examples to elucidate our viewpoint.

As depicted in Fig. 1(a), we have a benign behavior graph extracted from E3-THEIA, where the primary activity involves *Firefox* downloading a file named *dash* and executing it via *sshd*. Using this benign pattern as a baseline, we then analyze the detection outcomes shown in Fig. 1(b), (c), and (d). Fig. 1(b) reveals a *Firefox* node behavior pattern that diverges significantly from the previously learned benign pattern—this figure primarily shows *Firefox* engaged in downloading and reading files. Consequently, anomaly-based PIDSeS might flag this behavior as malicious, assuming that this benign pattern was not part of the learned dataset.

However, in Fig. 1(c), the structural features are analogous to those in Fig. 1(a), with both involving a process that downloads files externally and executes them through another process. Therefore, a model relying exclusively on structural features would fail to detect this attack pattern, leading to a false negative. Fig. 1(d) presents a benign behavior graph involving process-type nodes in reading and writing files, which, despite its substantial deviation from the benign pattern learned in Fig. 1(a) (where process nodes lacked external connections), might trigger an alert by anomaly-based PIDSeS. Yet, this is not indicative of an attack; rather, it is an alert triggered by the discrepancy from our learned benign behavior pattern, resulting in a false positive.

So, what criteria determine whether a *graph/path/node* is an attack? We offer the following insight: **A *graph/path/node* is identified as malicious not merely due to alterations in its structure, but also due to the presence of a propensity for malicious intent.** (We have re-analysis Fig. 1 by our methodology as detailed in Appendix C)

In this paper, we present MIRDETECTOR, an anomaly detection system that pioneers the incorporation of **malicious intent representations (MIR)** for nodes. MIRDETECTOR begins by constructing a knowledge graph from audit logs through straightforward and universally applicable steps (Zengy et al., 2022; Yang et al., 2023). Based on this knowledge graph and predefined policies, it then generates structural feature vectors and MIR feature vectors for nodes. MIRDETECTOR treats node attribute features as the third dimension of node features (where node attribute characteristics typically signal rarity, generally speaking, the attributes of malicious nodes are relatively rarer) and trains the three-dimensional node feature vectors using a Variational

Autoencoder (VAE) model, employing reconstruction error loss for detection. In essence, MIRDETECTOR assesses node maliciousness based on two criteria: (1) *the structural/attribute information of malicious nodes significantly deviates from that of benign nodes*, (2) *the malicious intent tendency of malicious nodes is considerably higher than that of benign nodes*. By analyzing and combining these detailed aspects of node behavior, attribute, and malicious intent representation, MIRDETECTOR enhances detection accuracy, reduces false positives commonly associated with traditional methods that rely solely on structural features, and also mitigates false negatives.

We have implemented MIRDETECTOR and carried out an extensive evaluation using the publicly accessible DARPA dataset. Our system was benchmarked against several state-of-the-art anomaly-based PID-Ses, such as ThreatTrace (Wang et al., 2022), Unicorn (Han et al., 2020), and KAIROS (Cheng et al., 2023). The evaluation results demonstrate that MIRDETECTOR's performance is competitive with these advanced systems, and in some respects, it even outperforms them, with a reduced false positive rate and enhanced detection capabilities.

In conclusion, the key contributions of this paper are outlined below:

- **Multidimensional Feature Integration.** We have synthesized features across three dimensions—structural features, attribute features, and MIR features. MIRDETECTOR's comprehensive consideration of these multiple dimensions addresses the prevalence of false positives associated with traditional anomaly detection methods that concentrate only on structural or attribute features, thereby improving the model's detection precision.
- **Incorporate Malicious Intent Representations.** We introduce a novel approach by being the first to include representations of node malicious intent in anomaly detection. MIRDETECTOR establishes a series of policies based on reports and white papers (Al-Mhiqani et al., 2020) to extract the MIR features of nodes, and propagates this representation through control and data flows.
- **Comprehensive Evaluation.** We assessed MIRDETECTOR using the publicly available DARPA benchmark dataset (Keromytis, 2018; Torrey, 2020) and compared its performance with current state-of-the-art anomaly-based PID-Ses. The evaluation outcomes show that MIRDETECTOR achieves high precision and recall rates, effectively detecting APT attacks. Compared to the baseline detection systems, its detection precision has been improved by up to 99%, and the recall rate has been increased by up to 68%. **Our code is publicly available on GitHub and can be accessed at <https://github.com/jynszbd/MIRDETECTOR..>**

2. Related work

MIRDETECTOR draws inspiration from two primary research domains: anomaly-based PID-Ses and rules-based PID-Ses.

Anomaly-based detection systems. Current anomaly-based PID-Ses predominantly detect intrusions by modeling the benign behavior of the system and detecting the deviation between the system activity and the modeled benign behavior. They can be classified according to detection granularity into graph-level, path-level, and node-level detection systems. Notable graph-level systems include StreamSpot (Manzoor et al., 2016), Unicorn (Han et al., 2020), ProGrapher (Yang et al., 2023), and SIGL (Han et al., 2021) etc. Streamspot converts provenance graphs into a fixed-size graphical sketch by tallying the occurrence of various substructures and applying the streamhash function, subsequently employing clustering for anomaly detection. Unicorn operates in a manner akin to Streamspot; it too tallies the occurrence of different substructures within provenance graphs and transforms them into a fixed-size vector using the HistoSketch (Yang et al., 2017), followed by real-time clustering for detection. ProGrapher deals with data in snapshot form, utilizes Graph2vec (Narayanan et al., 2017) for graph embedding, and then inputs the embeddings into TextRCNN (Lai et al., 2015) for

sequence prediction. SIGL employs graph LSTM (Hasan et al., 2009) to calculate the reconstruction loss for each node in a software installation graph, flagging the entire graph as anomalous if the total node loss of the graph surpasses a predefined threshold. These graph-level detection techniques primarily concentrate on learning the structural characteristics of graphs for anomaly detection, with principal detection methods including clustering, sequence prediction, and threshold-based judgment. Path-level detection systems, such as ProDetector (Wang et al., 2020) and Pogoda (Xie et al., 2018). ProDetector identifies anomalies by extracting paths from the provenance graph, converting these paths into vectors, and then performing detection based on clustering. Pogoda primarily assesses the scores of benign paths to determine thresholds, with edges scoring above a given threshold being flagged as anomalous paths. Node/edge-level detection systems, like ThreatTrace (Wang et al., 2022), ShadeWatcher (Zengy et al., 2022), and KAIROS (Cheng et al., 2023), Flash (Rehman et al., 2024) alert abnormal nodes within the system. ThreatTrace determines thresholds by learning the structural information around benign nodes, and nodes exceeding a given threshold are considered anomalous. ShadeWatcher, inspired by knowledge within the recommendation domain, detects anomalies by predicting the likelihood of interactions between two system entities. KAIROS uses a node attribute-based hierarchical feature hashing technique (Zhang et al., 2020) to encode node features and generate an initialization vector for nodes, then employs TGN (Rossi et al., 2020) to capture structural features in the provenance graph, and detects based on the reconstruction error of edge types. Flash employs a semantic encoder grounded in Word2Vec to capture the fundamental semantics of events within the traceability graph. It further integrates Graph Neural Networks (GNNs) to encode both local and global graph structures into node embeddings, thereby detecting deviations in node structure. Each of these anomaly detection methods has its unique characteristics and has significantly contributed to the advancement of our work. However, we observed a common limitation: they learn the structural features of benign graphs/paths/nodes. Deviations from the learned structure can lead to false positives, and they are also vulnerable to evasion attacks (Zügner et al., 2018; Wang and Gong, 2019). If malicious data resembles the learned structure, it can result in false negatives. This leads us to question whether learning only benign structural features is sufficient. Is a node considered abnormal solely due to changes in its structural features?

Rules-based detection systems. Rules-based PID-Ses have provided substantial inspiration. These systems (Milajerdi et al., 2019b; Hassan et al., 2019b; Milajerdi et al., 2019a; Hossain et al., 2017b; Zhu et al., 2023) use empirical knowledge to construct matching rules. When data from the provenance graph triggers predefined alert rules, a warning is issued. Homels (Milajerdi et al., 2019b) posits that while the specific attack steps of different APTs may greatly differ, high-level APT behaviors often adhere to the same kill chain, and have customized TTP (Tactics, Techniques, and Procedures) specifications based on MITRE's ATT&CK framework (Adversarial tactics, 2024). Sleuth (Hossain et al., 2017b), in response to the typical high-level objectives of attackers (1. Deploying and executing attacker code on the victim system; 2. Replacing or modifying critical files; 3. Exfiltrating sensitive data), has developed a series of tags and corresponding tag propagation strategies for detection. Nodoe (Hassan et al., 2019b) assigns an anomaly score to each edge based on the frequency of related events in the provenance graph, then uses a network diffusion algorithm to propagate these scores along adjacent edges in the graph, and generates aggregated anomaly scores for classification. Poirot (Milajerdi et al., 2019a) constructs a query graph based on expert knowledge from Cyber Threat Intelligence (CTI) and detects APT attacks by measuring the similarity between the provenance graph and the query graph, a method that can only detect known attack patterns. APTShield (Zhu et al., 2023) extracts several stages of attacks and attack characteristics based on the ATT&CK model and defines a series of rules to detect suspicious behaviors related to APT. In essence, in the detection approach of

rules-based PIDSeS, malicious behavior must meet certain malicious conditions (Conform to attack characteristics). This is a valuable approach. However, it is challenging to define comprehensive rules, and they struggle with detecting unknown attacks and scaling. Given these challenges, how can an attack be determined? How can the limitations of considering only structural features and the uncertainties introduced by relying solely on rule features be addressed? We are inspired by the detection philosophies of both approaches and propose that a node's classification as abnormal depends not only on changes in its structural/attribute features but also on its possession of certain malicious intent representations. We have demonstrated this through experiments.

3. Prior approaches & limitations

Anomaly-based PIDSeS are designed to identify deviations in system behavior patterns, potentially preventing issues before they escalate and detecting intrusions at the early stages of APT attacks, including unknown zero-day threats. These methods have garnered significant favor within the research community, sparking a surge in related studies. Currently, they are typically classified based on detection granularity into three categories: graph-level, path-level, and node/edge-level detection systems. However, we notice that across all detection levels, existing studies often focus solely on the structural information of the *graph/path/node* when modeling the system. For instance, Streamspot (Manzoor et al., 2016) and Unicorn (Han et al., 2020) identify statistically distinct substructures within the provenance graph, converting them into fixed-size vectors for subsequent clustering-based detection (Kaufman and Rousseeuw, 2009). SIGL (Han et al., 2021) constructs the causal context of each node using random walks and employs word2vec (Mikolov, 2013) to generate node vectors. ProGrapher (Yang et al., 2023) utilizes WL graph kernels (Shervashidze et al., 2011) to label each node and applies doc2vec (Le and Mikolov, 2014) to create low-dimensional vector representations, effectively modeling the d-hop structural features surrounding nodes. Provdetect (Wang et al., 2020) employs word2vec to generate feature vector representations of paths, while ThreaTrace (Wang et al., 2022) initializes node features based on the variety of neighboring nodes and edge types, refining the node feature vectors through GraphSAGE (Hamilton et al., 2017) by sampling and aggregating neighbor nodes. KAIROS (Cheng et al., 2023) employs hierarchical feature hashing techniques (Zhang et al., 2020) for initial node vector features and learns surrounding structural features using TGN (Rossi et al., 2006). All these anomaly detection systems rely on structural features (Some PIDSeS consider the attribute feature of nodes) for modeling, which can lead to model sensitivity, triggering alerts due to minor structural changes, and resulting in false positives.

MIRDETECTOR introduces a novel approach to anomaly detection. We posit that determining a node as malicious in anomaly detection depends on two factors: changes in its structural/attribute features and the representation of malicious intent. MIRDETECTOR is designed to detect malicious nodes based on these two criteria, offering a modeling strategy that addresses them. To a certain extent, MIRDETECTOR mitigates the overly sensitive, high false negative, and high false positive rate issues associated with traditional PIDSeS.

4. Threat model

Like much of the related work (Han et al., 2020; Milajerdi et al., 2019b,a; Cheng et al., 2023), we posit that attackers originate from external sources and may compromise system control by implementing communication backdoors, maintaining persistent connections within the system, and exfiltrating sensitive information. We exclude attacks conducted via covert traffic (such as covert channel attacks) from our considerations, as they are generally not recorded in kernel-level audit logs. As an anomaly detection system, MIRDETECTOR operates under

Table 1

Entities, their attributes, and event types.

Subject	Object	Events	Entity attributes
Process	Process	Fork, Clone	Process name
	File	Read, Write, Chmod, Exec	File name
	Socket	Sendto, Recvfrom	Src/Dst IP/Port

the assumption that it is trained exclusively on benign behaviors, implying that the host system remains unadulterated by attackers during this phase. Additionally, we presume that the system-level audit framework is impervious to tampering, meaning that attackers are unable to manipulate the audit logs' content directly. This integrity can be ensured through the use of tamper-resistant logging techniques (Paccagnella et al., 2020b,a).

5. System design

MIRDETECTOR is an innovative host-based intrusion detection system designed to detect APTs at the node level. Fig. 2 depicts the architecture of MIRDETECTOR, which comprises three primary components. We will provide a detailed description in 5.1, 5.2, 5.3

5.1. KG & MIR construction

5.1.1. Knowledge graph construction

MIRDETECTOR constructs a knowledge graph by ingesting system-provenance event streams collected by agents installed on monitored hosts (Pohly et al., 2012; Pasquier et al., 2017; Bates et al., 2015) (e.g., ETW, Audit, and CamFlow). The knowledge graph KG is defined through a set of vertices $V = v_1, v_2, v_3, \dots, v_n$ and edges $E = e_1, e_2, e_3, \dots, e_n$. Each edge in the KG is stored in the form of a triple $\langle src_id, dst_id, attr \rangle$, where *src_id* and *dst_id* are the ids of the source and destination nodes, respectively. The source node typically represents a process-type subject of the event, while the destination node represents the object being acted upon. The attribute *attr* includes details such as the operation type and timestamp. Furthermore, the MIRDETECTOR stores essential node attributes, including process names and associated command lines. Table 1 illustrates the types of subjects and objects MIRDETECTOR considers, their relationships, and the corresponding node attributes taken into account.

5.1.2. MIR flow construction

MIRDETECTOR posits that only process nodes possess a malicious intent representation within the provenance graph. This assumption is based on the fact that in the provenance graph, processes serve as the subjects executing operations within the system and are susceptible to malicious exploitation, whereas files and sockets do not exhibit tendencies toward malicious intent. In generating the MIR for nodes, our focus is primarily on process nodes as the principal entities.

To ascertain the MIR for each process node, MIRDETECTOR establishes a set of labels to define the intent, categorized into two main groups: feature labels and behavior labels, as depicted in Table 12 (Appendix A). Feature labels denote specific characteristics of a process. For instance, if a process node is linked to a socket node by an edge, it signifies that the process has a network connection, which we denote with the label PF1. Given that network data is often deemed untrusted, this process is assessed to have a higher level of malicious intent compared to others. Behavior labels, on the other hand, describe actions performed by the process, such as executing an untrusted file, which we label as PB1. Similarly, a process executing untrusted files is considered to have a heightened malicious propensity. To minimize the spread of label contamination, feature labels are predominantly used to describe behaviors preceding the transfer of subprocesses, carrying forward the semantic information from the parent process to the subprocess. Behavior labels, conversely, are used to describe

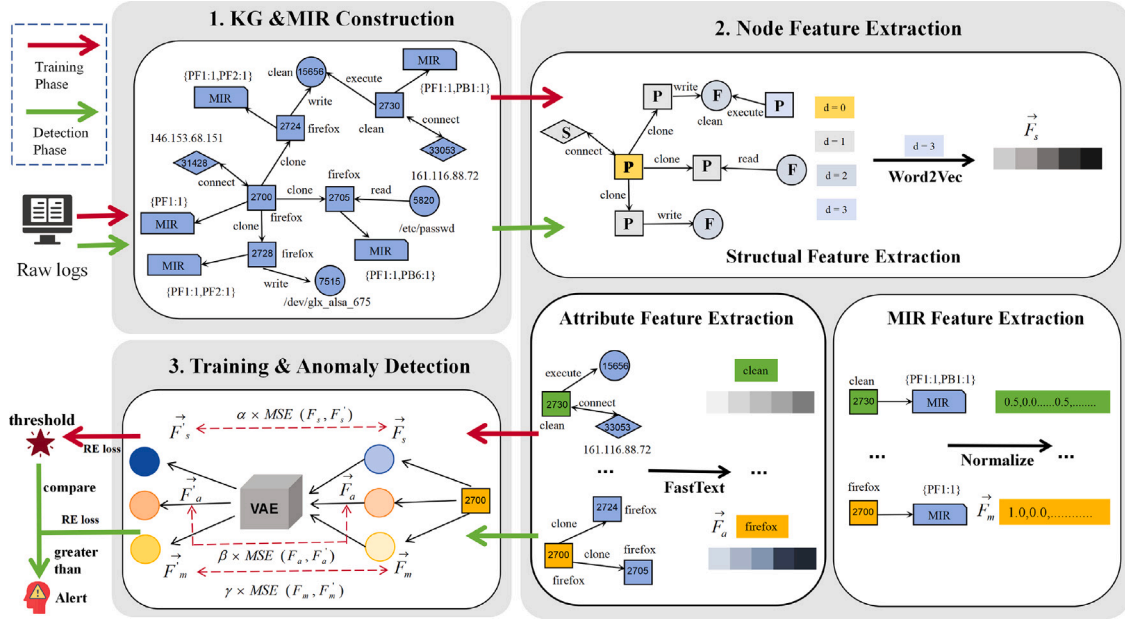


Fig. 2. Architecture Overview of MIRDETECTOR. The square in the figure represents the node of process type, the circle represents the node of file type, and the diamond represents the node of socket type.

process operations and are not transmitted to subprocesses through the system event flow.

To enhance the generation of MIR for process nodes, beyond defining process labels, we have also established file labels. MIRDETECTOR categorizes file labels into two types: confidential data and untrusted data, as illustrated in Table 13 (Appendix A). Confidential data is crucial to system security, and its exposure could potentially compromise the system. Untrusted data, on the other hand, is typically sourced from external entities.

MIRDETECTOR communicates contextual semantics between entities via control flow and data flow. The control flow mainly pertains to the relationship of process creation. We assume that when process A spawns another process B, process B inherits a degree of malicious intent equivalent to that of process A. The data flow, on the other hand, represents the trajectory of data movement. We distinguish data flow into two categories: untrusted data flow and high-value data flow. The untrusted data flow traces the path of data originating from external sources that are not considered reliable; the high-value data flow traces the path of data that is considered highly valuable or sensitive. As depicted in Table 11 (Appendix A), we outline the propagation rules for node labels between these flows.

Essentially, the propagation rules between entities, namely the transfer of control flow and data flow between them, equate to the propagation of malicious intent. By establishing node labels and their propagation rules, we can determine the MIR for each process node. These representations of malicious intent are presented in label form, with a node's MIR being the aggregate set of its labels.

5.2. Node feature extraction

5.2.1. Node structural feature extraction

To extract the structural features of nodes and enhance the algorithm's runtime performance, we utilize the WL (Weisfeiler-Lehman) graph kernel method for feature extraction. The WL graph kernel method (Shervashidze et al., 2011) is well-suited for our requirements as it can differentiate among various graph structures, boasts strong generalization capabilities, manages graphs of diverse sizes and types, and is computationally efficient. Specifically, we employ the WL graph kernel to capture the domain information of nodes, which, to a certain extent, also mirrors the structural attributes of these nodes. The detailed process is as follows:

- Step 1 (Initialization): We initialize the process by assigning the types of nodes as their respective labels and the types of events as the labels for edges (refer to Table 1).
- Step 2 (Iterative Process): In this step, for each node v , the WL graph kernel algorithm updates its label by considering the neighboring information of node v . During each iteration, the new label of a vertex is created by merging the vertex's label with the labels of the adjacent edges and nodes in a sequential manner.
- Step 3 (Feature Vector Construction): With the node labels generated from the previous steps, we employ the doc2vec skip-gram model (Le and Mikolov, 2014) to construct feature vectors for the nodes.

To characterize the structural features of a node v_i up to depth d , Step 2 is generally executed n times. Following this, the derived structural label of node v at depth d is utilized as input for the doc2vec skip-gram model to generate the node's d -hop structural features. To provide a clearer illustration of this process, we have included a detailed explanation in Appendix B, complete with specific examples.

5.2.2. Node attribute feature extraction

The semantic features of nodes also have a significant impact on detection. The research by Chen et al. (2021) indicates that the semantic features of nodes can greatly enhance the detection effect, with the highest improvement of up to 15% for autoencoders. Therefore, we use FastText (Joulin et al., 2016) to learn the semantic embedding feature information of nodes. Traditional FastText can only extract features at a single scale and is unable to capture both local and global semantic features simultaneously. Hence, we introduce a multi-scale feature extraction mechanism, which can extract semantic features at different scales through multi-layer n-gram feature extractors.

System logs typically contain multi-level semantic information. For example, the attributes of a *postgres* process node may include both local information (such as the specific content of command-line parameters: `-D /var/lib/postgresql/9.1/main -c config_file=/etc/postgresql/9.1/main/postgresql.conf`) and global information (such as the process name: `/usr/lib/postgresql/9.1/bin/postgres`). Through multi-scale feature extraction, both these different levels of semantic information can be extracted to more comprehensively capture the attribute features of

nodes. For the postgres node, the detailed feature extraction process is as follows:

Local feature extraction: Local features mainly focus on the specific content of command-line parameters, which usually contain specific details such as paths and configuration files. Short n-grams (such as 2-g and 3-g) are used to extract this local information. 2-g features: ['-D', '/var', 'var/lib', 'lib/postgresql', 'postgresql/9.1', '9.1/main', '-c', 'config_file', 'file=/etc', 'etc/postgresql', 'postgresql/9.1', '9.1/main', 'main/postgresql.conf']. 3-g features: ['-D /var/lib', 'var/lib/postgresql', 'lib/postgresql/9.1', 'postgresql/9.1/main', '-c config_file', 'config_file=/etc/postgresql', 'file=/etc/postgresql/9.1', 'etc/postgresql/9.1/main', 'postgresql/9.1/main/postgresql.conf'].

Global feature extraction: Global features mainly focus on the semantic categories of process names, which usually provide an overall description of the process. Long n-grams (such as 5-g and 7-g) are used to extract this global information. 5-g features: ['/usr/lib/postgresql/9.1/bin', 'lib/postgresql/9.1/bin/postgres']. 7-g features: ['/usr/lib/postgresql/9.1/bin/postgres']

Feature vector representation: The multi-scale features extracted are converted into vector representations through an embedding model (FastText). The corresponding command-line parameter embedding vectors are: f_2 (2-g embedding vector), f_3 (3-g embedding vector), and the process name embedding vectors are: f_5 (5-g embedding vector), f_7 (7-g embedding vector). The feature vectors of different scales are fused into a comprehensive feature vector, so the attribute feature vector of the node is $f_a = f_2 + f_3 + f_5 + f_7$.

5.2.3. Node MIR features extraction

We can derive the MIR of nodes, which are displayed as labels. During the feature extraction phase, we initially determine the various types of labels (e.g., *PB1*, *PF1*, etc.), denoted as N_n . We establish a label mapping function that assigns labels to an integer range from 0 to N_n-1 , while also tallying the number of distinct labels. For instance, at a given moment, after the MIR flow construction, a node process might possess the following labels: {PF1: 5, PF2: 2, PB1: 1, PB6: 3}. Consequently, its corresponding node MIR feature vector would be: [5, 2, 0, 0, 0, 1, 0, 0, 0, 0, 3, 0, 0], where each dimension represents a distinct malicious feature, and the numbers within the same dimension indicate varying degrees of malicious intent. To mitigate the impact of feature variance across different dimensions of the model, we normalize the node's MIR features.

5.3. Model training and anomaly detection

We have employed a VAE for our model. The VAE model, known for its efficiency and lightweight performance, maps input data to a low-dimensional latent space via an encoder and reconstructs the data using a decoder. To adapt the VAE to our detection methodology, we have enhanced the VAE by introducing a multi-objective optimization strategy that simultaneously optimizes the reconstruction errors of the structural features, attribute features, and MIR features of the nodes. Specifically, we incorporate multiple loss functions to compute the reconstruction error for each feature dimension separately, and then combine them into a comprehensive loss function. As outlined in Section 5.2, we extract three dimensions of features for each node: structural features (F_s), attribute features (F_a), and MIR features (F_m). These three sets of features are fed into the VAE model for separate training, with loss calculated using distinct loss functions, yielding the reconstructed vectors F'_s , F'_a , and F'_m .

To compute the reconstruction loss for each node, we utilize three parameters: α , β , and γ . These parameters signify the relative influence of a node's structural features, attribute features, and MIR features on detection, respectively. In our evaluation (Section 6.4), we explored the impact of varying the values of α , β , and γ on detection performance. The reconstruction loss for a node is given by the formula in Eq. (1).

$$RE = \alpha \times MSE(F_s, F'_s) + \beta \times MSE(F_a, F'_a) + \gamma \times MSE(F_m, F'_m) \quad (1)$$

Our detection criterion is based on the premise that anomalous nodes are more challenging to accurately represent in vector space, leading to higher loss values during reconstruction attempts. MIRDETECTOR categorizes changes in a node into three dimensions: alterations in the node's structural features, attribute features, and MIR features. The structural features capture the behavior of the node, the attribute features indicate the node's rarity, and the MIR features signify the node's potential harm. MIRDETECTOR learns the normal patterns of these three characteristics for benign nodes, employs standardized MSELoss to quantify the discrepancy between the input and reconstructed vectors, and utilizes the reconstruction error RE to denote the MSELoss value between the input and reconstructed vectors. MIRDETECTOR posits that a higher reconstruction loss RE indicates a greater deviation of the node from its normal pattern, encompassing not only structural divergence but also malicious intent and attribute deviation. By taking a holistic approach that considers all three aspects for detection, MIRDETECTOR can accurately pinpoint malicious nodes and further mitigate the false positives and false negatives associated with traditional methods that focus on structural/attribute features.

6. Evaluation

We developed the MIRDETECTOR prototype using Python 3.9, leveraging the gensim (Gensim, 2024) library for Doc2Vec and FastText implementations, and utilizing networkx (Networkx, 2024) for constructing the knowledge graph. The VAE model within MIRDETECTOR was developed using PyG (Pytorch, 2024). MIRDETECTOR gathers data from monitored hosts through agents, with the collected data formatted uniformly with other detection systems (Han et al., 2020; Zengy et al., 2022; Wang et al., 2022; Cheng et al., 2023; Wang et al., 2020). Our experiments were conducted on a server running Ubuntu 18.04 64-bit, equipped with an Intel(R) Xeon(R) Silver 4214R CPU @ 2.40 GHz and 256 GB of RAM.

MIRDETECTOR was evaluated using five publicly accessible datasets, with a detailed description of these datasets provided in Section 6.1. To benchmark MIRDETECTOR's performance against other anomaly-based PIDSeS, we selected three representative systems—Unicorn (Han et al., 2020), ThreatTrace (Wang et al., 2022), and KAIROS (Cheng et al., 2023)—as comparative baselines. As a novel approach to anomaly detection, our evaluation aimed to address the following research questions:

- **RQ 1:** Is MIRDETECTOR capable of accurately detecting anomalies in systems currently under attack? (Section 6.2)
- **RQ 2:** How does MIRDETECTOR's performance stack up against the most advanced anomaly detection systems? (Section 6.3)
- **RQ 3:** How do hyperparameters impact MIRDETECTOR's detection effectiveness? (Section 6.4)
- **RQ 4:** How significant are the MIR features of nodes for attack detection? (Section 6.5)
- **RQ 5:** What is the detection efficiency of MIRDETECTOR? (Section 6.6)

6.1. Dataset

To thoroughly assess the detection capabilities of MIRDETECTOR against APT attacks, a comprehensive, long-term dataset is essential. For this reason, we have chosen the DARPA Transparent Computing (TC) dataset. The DARPA TC project is a cybersecurity research initiative funded by the Defense Advanced Research Projects Agency (DARPA) in the United States.

The DARPA TC project conducted five extensive red-team versus blue-team cyber warfare exercises from October 2016 to May 2019, with participation from various technical teams. Throughout these exercises, the red team executed a range of attacks on enterprises exploiting vulnerabilities such as the *nginx backdoor*, *firefox backdoor*,

Table 2
Related dataset information.

DataSet	# of nodes	# of edges	% of process nodes	# of attack process nodes	% of attack process nodes
E3-THEIA	1 317 368	25 443 381	278 363	37	0.01329
E3-CADETS	613 713	21 781 779	224 146	2	0.00089
E3-TRACE	43 489 627	252887381	39 283 148	13	0.00003
E5-THEIA	2 309 372	139744718	1 264 440	18	0.00142
E5-CADETS	15 337 462	983717206	6 683 933	76	0.00114

Table 3
DARPA data used for training and testing.

Datasets	Training data (yyyy-mm-dd)	Test data (yyyy-mm-dd)
DARPA-E3-THEIA	2018-04-03/04/05	2018-04-10/12
DARPA-E3-CADETS	2018-04-02/03/04	2018-04-06
DARPA-E3-TRACE	2018-04-08	2018-04-10
DARPA-E5-THEIA	2019-05-08/09	2019-05-15
DARPA-E5-CADETS	2019-05-09	2019-05-17

and browser extensions, while also engaging in benign activities like web browsing, email checking, and SSH logins. We selected the datasets from the third (E3) and fifth (E5) exercises for our analysis because they are publicly accessible (Keromytis, 2018; Torrey, 2020) and have been widely utilized in the field (Han et al., 2020; Wang et al., 2022; Han et al., 2021; Cheng et al., 2023). Table 2 presents detailed statistical information from these datasets and other sources. Table 3 details the training dataset and the test dataset we use.

Data Labeling We derived the ground truth for the attacks from the DARPA-provided truth file (Darpa, 2020). Attacks are confined to specific time frames, prompting us to manually label processes associated with these attacks based on the details in the DARPA report and the actual DARPA dataset. Our analysis revealed that during an attack, a process often spawns multiple child processes to fulfill various objectives of the attack. For instance, in the THEIA dataset, at 1:41 PM on April 10, 2018, during an attack, the attacker cloned several Firefox processes to exploit a Firefox vulnerability and gain access to the target host. Some of these processes were utilized to write profile files, while others wrote clean files. We designated processes engaged in specific attack activities as attack processes and also marked their parent processes, which spawned these attack processes, as attack processes.

6.2. Detection performance

In our evaluation, we employed four key metrics: True Positives (TP), False Positives (FP), True Negatives (TN), and False Negatives (FN). TP refers to the instances that are actually malicious and correctly identified as such. FP denotes the instances that are benign but incorrectly classified as malicious. TN represents the instances that are benign and correctly identified as benign. FN indicates the instances that are malicious but incorrectly classified as benign. The formulas for calculating Accuracy, Precision, Recall, and F1 are as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \quad (2)$$

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (5)$$

Table 4 displays the Accuracy, Precision, Recall, and F1 scores achieved by MIRDETECTOR across all datasets. These metrics are calculated

Table 4
Results by node ID.

DataSet	TP	FP	TN	FN	Precision	Recall	Accuracy	F1
E3-THEIA	34	5	18 769	3	0.87	0.92	0.99	0.89
E3-CADETS	2	0	19 808	0	1.00	1.00	1.00	1.00
E3-TRACE	12	3	567 647	1	0.80	0.92	0.99	0.86
E5-THEIA	16	4	27 430	2	0.99	0.80	0.88	0.84
E5-CADETS	55	1	68 119	21	0.98	0.72	0.99	0.83

at the level of individual process nodes. As detailed in Section 6.1 regarding data labeling, we manually annotated malicious nodes by the ground truth. Utilizing the four computational metrics previously discussed, we derived the respective Accuracy, Precision, Recall, and F1 scores.

As observed in Table 4, MIRDETECTOR achieves commendable detection results at the node level, demonstrating the ability to identify attacking nodes among a multitude of nodes. On the E3-THEIA dataset, we identified 37 attack nodes, a result of multiple nodes forked by processes like Firefox and profile to accomplish related attack operations. MIRDETECTOR only missed 3 nodes in the E3-THEIA dataset, which were Firefox nodes. Upon review, the missed detection was attributed to these nodes' structural features and MIR features being highly similar to those of normal nodes. We consider this level of false negatives to be acceptable. On one hand, the missed Firefox nodes exhibit a low degree of malicious intent and structural features akin to normal nodes, indicating a minimal level of maliciousness and risk. On the other hand, our labeling is based on ID. In reality, within this dataset, Firefox-type nodes have already been flagged, and we have issued the corresponding attack alerts. To prevent further harm from the process, all Firefox processes could be terminated to halt the attack progression. Similarly, we found that nodes in the E5-THEIA and E3-TRACE datasets also experienced the same situation, with false positives and false negatives occurring due to nodes sharing the same attributes but different IDs. We re-evaluated the model's test performance in Table 5 based on the actual attributes of the nodes (without distinguishing node IDs).

In the E3-CADETS dataset, the attack was relatively straightforward, with Nginx and vUgefal nodes each using a single node to complete the attack operations, without forking multiple nodes for the attack. This results in more complete MIR features and structural features of the nodes, leading to better model detection outcomes. It should be noted that in the E3-CADETS dataset, we only marked two attack nodes, differing from the number of process nodes marked in KAIROS (Cheng et al., 2023) and the ground truth file (Darpa, 2020). This discrepancy arises because we did not identify sshd nodes that met the conditions during the attack period. We also found that KAIROS did not detect nodes with sshd attributes in the final detection attack graph. We believe this is an issue that occurred during data collection, resulting in no sshd attribute nodes in the final dataset. MIRDETECTOR performed not well in the E5-CADETS dataset. Upon inspection, we found that the missed nodes were cat and find nodes. Further analysis of the original dataset revealed that cat and find nodes forked by the bash node did not perform any operations. Neither the node's MIR features nor the structural features showed any significant changes, which led to the MIRDETECTOR missing the detection of these nodes. We believe that whether nodes such as cat and find should be defined as malicious nodes is open to debate, as they, although forked by the malicious node bash, do not pose a threat in themselves because they did not perform

Table 5
Results by node attribute.

DataSet	TP	FP	TN	FN	Precision	Recall	Accuracy	F1
E3-THEIA	41	1	18769	0	0.98	1.00	0.99	0.99
E3-CADETS	2	0	19808	0	1.00	1.00	1.00	1.00
E3-TRACE	16	0	567647	0	1.00	1.00	1.00	1.00
E5-THEIA	20	2	27430	0	0.91	1.00	0.99	0.95
E5-CADETS	60	1	68119	16	0.98	0.79	0.99	0.88

Table 6
Experimental results of UNICORN and MIRDETECTOR.

Datasets	System	Precision	Recall	Accuracy	F1
E3-CADETS	Unicorn	0.98	1.00	1.00	0.99
	MIRDETECTOR	1.00	1.00	1.00	1.00
E3-THEIA	Unicorn	1.00	1.00	1.00	1.00
	MIRDETECTOR	1.00	1.00	1.00	1.00

any operations. Therefore, the classification of whether such nodes are malicious requires further determination. However, we consider this level of false negatives to be acceptable. On one hand, due to the low risk of these nodes, and on the other hand, security analysts can further identify the corresponding *cat*, *find*, and other nodes based on the alerted *bash* nodes.

6.3. Comparison study

To assess the effectiveness of MIRDETECTOR in comparison with the state-of-the-art anomaly detection systems, we chose Unicorn (Han et al., 2020), ThreatTrace (Wang et al., 2022), and KAIROS (Cheng et al., 2023) as our benchmarks. The rationale for selecting these systems is their recency in research, open-source availability, and reproducibility.

6.3.1. UNICORN vs. MIRDETECTOR

UNICORN ingests labeled real-time provenance graphs and constructs a streaming histogram in memory, capturing a comprehensive record of the system's execution history. Each histogram element represents a unique subgraph structure. UNICORN utilizes the HistoSketch (Yang et al., 2017) technique to transform the histogram into a graph sketch, maintaining the normalized Jaccard similarity (Chum et al., 2008) between two graph histograms. It then employs the K-medoids algorithm (Kaufman and Rousseeuw, 2009) for clustering purposes. UNICORN identifies anomalies based on two primary criteria: sketches that do not align with existing clusters and invalid transitions between clusters. UNICORN operates at the graph level, categorizing entire graphs as either benign or indicative of an attack. To ensure a consistent evaluation approach, we followed the same assessment protocol as UNICORN. We segmented the knowledge graph into subgraphs matching the Unicorn's size and fed process nodes into the model for detection. The outcome still pinpoints anomalous process nodes. In our evaluation, graphs containing anomalous process nodes were considered attacked graphs. We did not utilize the dataset from Manzoor (2016) due to its lack of relevant attribute information for nodes, which is critical for constructing the MIR of nodes. Since Unicorn was primarily tested on the DARPA E3 dataset, to prevent hyperparameter tuning effects, MIRDETECTOR was also not assessed on the DARPA E5 dataset. We observed that the ClearScope dataset exhibited some data discontinuity; for unknown reasons, behaviors (CLONE/FORK/EXECUTE) in ClearScope impeded the flow of inter-process information, which is vital for our MIR construction. Thus, we excluded the ClearScope dataset from our evaluation and focused solely on the DARPA E3 (CADETS, THEIA) datasets.

Experimental Results: Table 6 presents the comparative experimental outcomes of MIRDETECTOR and Unicorn across two datasets. As observed from Table 6, MIRDETECTOR either outperforms Unicorn

Table 7
Experimental results of ThreatTrace and MIRDETECTOR.

Datasets	System	Precision	Recall	Accuracy	F1
E3-THEIA	ThreatTrace	0.003	0.24	0.84	0.006
	MIRDETECTOR	0.87	0.92	0.99	0.89
E3-CADETS	ThreatTrace	0.003	0.80	0.88	0.007
	MIRDETECTOR	1.00	1.00	1.00	1.00
E3-TRACE	ThreatTrace	3.73e-05	1.00	0.18	7.47e-05
	MIRDETECTOR	0.80	0.92	0.99	0.86

or matches its high level of performance on all datasets. Unlike Unicorn, which lacks fine-grained detection capabilities and typically triggers alerts for graphs with a multitude of nodes, MIRDETECTOR is capable of precisely identifying nodes involved in attacks. This precision significantly eases the complexity of attack investigation.

6.3.2. THREATTRACE vs. MIRDETECTOR

ThreatTrace ingests provenance data and employs a customized inductive graph neural network framework, GraphSAGE (Hamilton et al., 2017), to learn the benign roles of nodes within the provenance graph, thereby detecting subtle abnormal behaviors at the node level. Similarly, we do not use the Manzoor dataset. Since ThreatTrace has only released the relevant code and ground truth files for the DARPA3 dataset, to avoid the impact of parameter tuning and data labeling, we also compare only within the DARPA3 (CADETS, THEIA, and TRACE) dataset when comparing with ThreatTrace. As MIRDETECTOR relies on processes as computational metrics, we replicated ThreatTrace without altering its parameters. We processed the ground truth file provided by ThreatTrace, excluding file nodes and socket nodes, and focused solely on process nodes. During the evaluation phase, we considered only the process nodes alerted by ThreatTrace. Subsequently, we employed the same computational metrics as MIRDETECTOR for our evaluation to ensure the fairness and consistency of the assessment results.

Experimental Results: Table 7 presents the comparative experimental outcomes of MIRDETECTOR and ThreatTrace on three datasets. Our analysis revealed that ThreatTrace produced a significant number of false positives. In the THEIA dataset, ThreatTrace identified a total of **9795** process nodes, while the process-level ground truth marked only 21. The number of alerts issued by ThreatTrace was **1532**, with a mere 5 of these alerted nodes being present in the ground truth, leading to a precision rate of just 0.003. Strikingly, in the TRACE dataset, ThreatTrace flagged 8 GroundTruth nodes, yet the alert count soared to **214,055**. Navigating security analysis amidst such an overwhelming number of false positives is exceedingly challenging. Conversely, MIRDETECTOR demonstrated the ability to accurately pinpoint threat nodes among a vast array of nodes, yielding a considerably lower rate of false positives. This capability makes MIRDETECTOR a more reliable tool for security analysis, as it significantly reduces the noise associated with false alarms, allowing for more efficient and accurate threat detection.

6.3.3. KAIROS vs. MIRDETECTOR

KAIROS captures the temporal evolution of structural changes within provenance graphs and detects anomalous system events at the time window level using a graph neural network encoder-decoder framework. Our observations indicated that when comparing KAIROS with ThreatTrace, it leveraged anomalous nodes within suspicious time windows for node-level analysis. Consequently, we also evaluated KAIROS's detection effectiveness at the node level using this methodology. To ensure a consistent evaluation approach, akin to our methodology with ThreatTrace, we utilized the original KAIROS code without altering its parameters, excluding file and socket-type nodes and focusing solely on process-type nodes for our assessment. It is important to note that KAIROS considers nodes with identical attributes but distinct IDs as the same node during evaluation, hence, we adhered to the same evaluation criteria as KAIROS. KAIROS was tested on the

Table 8
Experimental results of KAIROS and MIRDETECTOR.

Datasets	System	TP	FP	TN	FN	Precision	Recall	Accuracy	F1
E3-CADETS	KAIROS	1	1	19 806	2	0.50	0.40	0.99	0.40
	MIRDETECTOR	2	0	19 807	1	1.00	0.67	0.99	0.80
E3-THEIA	KAIROS	3	0	18 805	3	1.00	0.50	0.99	0.67
	MIRDETECTOR	6	0	18 805	0	1.00	1.00	1.00	1.00
E5-CADETS	KAIROS	1	0	68 191	4	1.00	0.20	0.99	0.33
	MIRDETECTOR	3	1	68 190	2	0.75	0.60	0.99	0.67
E5-THEIA	KAIROS	0	0	27 449	3	/	0.0	0.99	/
	MIRDETECTOR	3	2	27 447	0	0.60	1.0	0.99	0.75

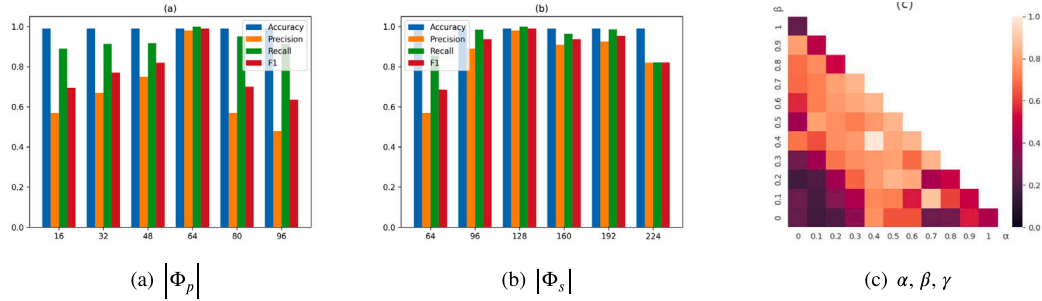


Fig. 3. The effects of the node attribute embedding dimensions $|\Phi_p|$, the node structural embedding dimension $|\Phi_s|$, and the proportion changes of α , β , and γ on the detection results.

DARPA3 and DARPA5 datasets. Owing to data discontinuity issues within the CLEARSCOPE dataset, we opted not to evaluate it and limited our evaluations to the CADETS and THEIA datasets.

Experimental Results: Table 8 presents the comparative experimental results of MIRDETECTOR and KAIROS across four datasets. We analyzed the process nodes in KAIROS's anomaly queue at the process node level, using the attack diagrams provided by KAIROS in their documentation and the ground truth file for evaluation. Our findings indicate that KAIROS's performance at the process node level was suboptimal. In the E3-THEIA dataset, there are six processes in the ground truth: *Firefox*, *Crashreporter*, *Profile*, *Clean*, *Gtcache*, and *Mail*. However, KAIROS's anomalous queue only included *Firefox*, *Crashreporter*, and *Profile* nodes. Similarly, in the E5-CADETS dataset, only the *nginx* node was identified. In the E3-CADETS dataset, both MIRDETECTOR and KAIROS failed to detect the *sshd* node, which we previously attributed to a data collection issue (Section 6.1). In the E5-CADETS dataset, both KAIROS and MIRDETECTOR underperformed; MIRDETECTOR missed the *cat* and *found* nodes (Section 6.2), while KAIROS only alerted the *nginx* node. In the E5-THEIA dataset, no nodes were alerted within KAIROS's suspicious time windows. This might be an issue originating from the original code in KAIROS. Overall, as seen in Table 8, MIRDETECTOR's detection effectiveness at the node level significantly outperforms KAIROS, as it is capable of identifying nodes with malicious attack tendencies among a large number of nodes.

6.4. Hyperparameter impact on performance

In this section, we analyze the influence of several hyperparameters on detection effectiveness. For efficiency, we have chosen Day 10 and Day 12 from the E3-THEIA dataset as our evaluation datasets. We adjust each hyperparameter individually and document its effect on both detection performance and runtime; the results of these experiments are depicted in Fig. 3.

Node attribute embedding dimension ($|\Phi_p|$). The node attribute embedding translates node attributes into a low-dimensional space, where similar attribute information should be proximate. As shown in Fig. 4(a), an embedding dimension that is too small fails to encode these features effectively. Conversely, a dimension that is too large results in sparse features, adversely affecting detection performance

and precision. After experimentation, we determined that an ideal dimension is $|\Phi_p| = 64$.

Node Structural Embedding Dimension ($|\Phi_s|$). Node structural embedding captures the surrounding domain information of nodes. An insufficient dimension for $|\Phi_s|$ means the features are too limited to represent domain information effectively, leading to a significant drop in detection precision. On the other hand, an overly large dimension for $|\Phi_s|$ can lead to sparse features, impacting detection effectiveness. Fig. 4(b) shows how MIRDETECTOR's detection metrics vary with different settings of $|\Phi_s|$. The results suggest that an optimal dimension is achieved when $|\Phi_s| = 128$.

The impact of α , β , and γ . α , β , and γ represent the relative influence of a node's structural features, attribute features, and MIR features on detection, respectively. Node structural features reflect its behavior, attribute features indicate the node's rarity, and MIR features signal its potential harm. MIRDETECTOR assesses node maliciousness based on changes in these three aspects, making their relative impact on detection crucial. To demonstrate the impact of these factors on our model's effectiveness, we present the results in a heatmap, where the horizontal axis represents α values, the vertical axis represents β values, and γ is implicitly represented as $\gamma = 1 - \alpha - \beta$. The heatmap values correspond to the F1 scores for each combination. The graph indicates that optimal detection results are achieved when $\alpha = 0.4$, $\beta = 0.4$, and $\gamma = 0.2$. Additionally, we found that node attribute features are pivotal for enhancing detection effectiveness. Without node attribute features, no matter how α and β are adjusted, the model's detection performance cannot be further improved.

6.5. The importance of MIR feature in detection

To ascertain the significance of node MIR features in detection, we conducted additional ablation experiments on the DARPA3 dataset to assess the detection capabilities of MIRDETECTOR without incorporating the MIR features of nodes. In these experiments, we eliminated the MIR features of nodes, retained their attribute and structural features, and adjusted the parameters for the structural features. Specifically, we set parameter α to the sum of its original value and β and set β to 0, before retraining and testing the model. This evaluation focused primarily on node attributes, considering the correct detection of the

Table 9
Results with MIR vs. without MIR.

Datasets	Detection with MIR			Detection without MIR		
	Precision	Recall	Accuracy	Precision	Recall	Accuracy
E3-CADETS	1.00	1.00	1.00	0.002	1.00	0.739
E3-THEIA	0.98	1.00	0.99	0.103	0.917	0.979
E3-TRACE	1.00	1.00	1.00	0.471	1.00	0.999

Table 10
Overhead of each component.

Component	Mean duration
Data processing	4.42 min
Node structural feature extraction	17.565 min
Node attribute feature extraction	3.94 s
Node MIR feature extraction	9.63 s
Model training	34.21 min
Anomaly detection and malicious node generation	10.32 s

attributes of corresponding malicious nodes as a successful identification of malicious nodes. The results of these experiments are presented in Table 9. We observed that the omission of node MIR features led to a substantial increase in false positives during MIRDETECTOR's detection process. In the E3-CADETS dataset, around 7010 nodes triggered alerts, but only 11 of these were valid *nginx* nodes, with just 1 being a *vUgefal* node. In the E3-THEIA dataset, approximately 426 nodes were alerted, with only the *clean* and *profile* nodes being effectively identified, while nodes such as *gtcache* were overlooked. These findings underscore the critical role of node malicious intent features and our approach's ability to significantly reduce false positives and false negatives.

6.6. Detection efficiency

In this section, we evaluate the overhead of each component of MIRDETECTOR and present the results in Table 10.

We performed several iterations using the DARPA3 THEIA dataset (data days 03,04,05,09,10,12) and computed the average to derive the results shown in the table. On average, MIRDETECTOR takes approximately 5 min to process the system logs for a single day's dataset, and it can complete the generation of its node attribute features and malicious intent features within 10 s. Owing to the lightweight VAE model employed for training, MIRDETECTOR only requires an average of 35 min to train for 100 epochs. It is worth noting that these training times were achieved using a server with a single CPU core; thus, the overhead is expected to decrease when utilizing a GPU or a distributed training environment.

7. Discussion

Online Detection. The timeliness of APT attack detection systems is a critical attribute, as it facilitates the early identification and effective mitigation of potential cyber threats at the initial stages of an attack. This proactive approach significantly expedites system recovery and minimizes the potential damage caused by the attack. Online detection tools are essential in this context, and MIRDETECTOR, with its optimized design and robust capabilities, effectively meets the demands of real-time monitoring. Specifically, MIRDETECTOR employs a lightweight FastText model to efficiently process the continuous generation of node attribute vectors, ensuring high operational efficiency even when handling large-scale data. Additionally, MIRDETECTOR's MIR feature supports streaming generation, enabling real-time responsiveness to dynamic network environments. Its method for extracting structural features is consistent with existing online detection systems (Yang et al., 2023; Han et al., 2020). To further enhance its online detection capabilities, MIRDETECTOR utilizes a lightweight Variational Autoencoder (VAE) model for detection, ensuring both efficiency and accuracy in its online detection process.

Evasion Attacks. Attackers may attempt to deceive intrusion detection systems by concealing their malicious activities within normal system operations. However, this strategy presents significant challenges. To successfully emulate benign behavior, attackers must meticulously plan their strategies to ensure that their malicious actions appear innocuous in log records while still achieving their attack objectives. This requires a deep understanding of the target system's normal operations and precise knowledge of the mechanisms of PIDSes. Research (Alshamrani et al., 2019; Xu et al., 2024) has shown that existing graph-based adversarial attacks cannot effectively evade PIDSes due to the rich structural features and temporal constraints of provenance graphs. A recent study (Goyal et al., 2023) proposed a method to evade PIDSes by automatically modifying system logs, but this approach still faces many limitations in practical applications. We posit that evading detection by MIRDETECTOR through such means is not straightforward. Specifically, MIRDETECTOR operates at the node level rather than the graph level, rendering graph-structure-disrupting methods less effective against it. Moreover, MIRDETECTOR detects not only structural changes but also leverages node MIR features for comprehensive analysis. If attackers attempt to achieve their goals, they will inevitably leave traces of the attack, which will manifest as certain malicious tendencies in the provenance graph. By focusing on both structural changes and MIR features, MIRDETECTOR can effectively identify attack behaviors, thereby thwarting evasion attempts. To empirically validate the effectiveness of MIRDETECTOR against evasion attacks, we attempted to use the public DARPA E3-THEIA dataset released by previous research (Goyal et al., 2023). However, we found that the dataset lacks detailed information about relevant nodes and edges (such as node attribute information and edge types), which is crucial for obtaining node MIR features. Therefore, we were unable to conduct experimental testing on this dataset, and we will investigate and refine the relevant mechanisms in our future work.

Attack Chain Generation. In the realm of anomaly detection, merely alerting the corresponding attack nodes is insufficient. Generating attack chains is essential for comprehending the system's anomalous behavior. The generation of attack chains not only aids security analysts in gaining a more comprehensive understanding of the attack paths and the attackers' intentions but also significantly reduces their workload and accelerates the identification and elimination of false positives. MIRDETECTOR excels in this regard, as it supports the generation of attack chains based on the MIR flow we have constructed. Specifically, MIRDETECTOR can utilize the alerted process nodes and the previously constructed MIR flow to identify the transmitted MIR flow. It then integrates the alerted nodes and reconstruction loss, employing Shortest Path Trees (STP) (Hwang and Richards, 1992; Imase and Waxman, 1991) to construct an attack investigation graph. This attack investigation graph provides security analysts with intuitive and detailed information about the attack paths, thereby facilitating more efficient attack analysis and response.

8. Conclusion

We propose MIRDETECTOR, a novel anomaly detection system for APTs. Unlike conventional anomaly-based methods that often rely on singular or limited feature sets, MIRDETECTOR integrates structural and attribute features of nodes with representation of malicious intent, providing a more holistic representation of potential threats, significantly reducing false positives and false negatives. Our comprehensive evaluation of MIRDETECTOR reveals its superior precision and recall rates, which outperform existing state-of-the-art technologies. MIRDETECTOR demonstrates a remarkable ability to discern malicious nodes within large-scale networks while minimizing erroneous classifications of benign nodes. Its lightweight design ensures efficient, real-time detection with minimal performance overhead. Overall, MIRDETECTOR offers a valuable solution to enhance cybersecurity against complex APT attacks.

Table 11
Definition of propagation rules.

Subject	Event	Object	Label receptor	Description
Process	Fork clone	Process	Subject	When a process forks a child process, the labels of the parent process are passed to the child process.
Process	Read Execute Load	File	Subject	If a process reads or loads a file, then the process is tagged with the corresponding file attribute labels (PB5, PB6).
Process	Write	File	Object	If a process writes to a file, then the file is tagged with the file labels corresponding to the labels carried by the process (FC5).
Process	Sendto recvfrom	Socket	Subject	If a process has a network connection, then the process is tagged with the corresponding label (PF1).

CRedit authorship contribution statement

Hongmei Li: Writing – original draft, Visualization, Methodology, Conceptualization, Validation, Investigation, Resources, Data curation. **Tiantian Zhu:** Writing – original draft, Methodology, Validation, Funding acquisition, Writing – review & editing, Supervision, Formal analysis. **Jie Ying:** Resources, Writing – original draft, Methodology, Software, Investigation. **Tieming Chen:** Writing – review & editing, Funding acquisition, Supervision, Resources. **Mingqi Lv:** Supervision, Investigation, Writing – review & editing, Funding acquisition. **Jian-Ping Mei:** Supervision, Writing – review & editing, Funding acquisition, Validation. **Zhengqiu Weng:** Writing – review & editing, Formal analysis, Supervision, Funding acquisition. **Lili Shi:** Supervision, Methodology, Writing – review & editing, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the following grants: National Natural Science Foundation of China under Grant No. U22B2028, 62372410, 62002324, and 62276234. Zhejiang Provincial Natural Science Foundation of China under Grant No. LQ21F020016, ZCLZ24F0202, and LZ23F020011. Key R&D Projects in Zhejiang Province under Grant No. 2021C01117. Major Program of Natural Science Foundation of Zhejiang Province under Grant No. LD22F020002. The Fundamental Research Funds for the Provincial Universities of Zhejiang under Grant No. RF-A2023009. Wenzhou Key Scientific and Technological Projects under Grant No. ZG2024007. Wenzhou Basic Scientific Research Projects under Grant No. G2024033. Wenzhou Basic Scientific Research Project in 2024: “Research on the Application of Contrastive Learning Blink Detection in Industrial Production Safety” under Grant No. R20240138.

Appendix A. Relevant tables

To enhance readability, we have moved some tables to [Appendix A](#). Specifically, [Tables 11](#), [12](#), and [13](#) respectively provide the relevant definitions of process labels, file labels, and propagation rules as mentioned in [Section 5.1](#).

Appendix B. A example of node structural feature extraction

To more clearly illustrate the generation process of vertex structure labels, we have provided a detailed description of the structure label generation process for vertex *Process1* in [Appendix B](#), and intuitively demonstrated it through [Fig. 4](#).

At depth $d = 0$, the initial label for *Process1* is simply $[(Process)]$.

At depth $d = 1$, we examine the one-hop neighborhood of *Process1*. Now, the label for *Process1* becomes $[(Process), (Process), (Socket), Clone, Connect)]$.

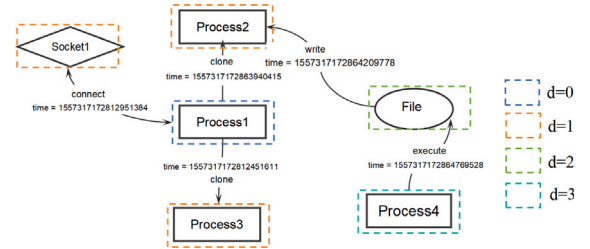


Fig. 4. The process of generating structural labels for *Process1*.

At depth $d = 2$, we look at the two-hop neighborhood around *Process1*. The labels of the neighboring nodes are updated to incorporate the labels of their immediate one-hop neighbors. For instance, *Process2*'s label is updated to $[(Process, File, Process, Clone, Write)]$. Then, *Process1* refines its label by incorporating its current label and the labels of its surrounding neighbors. At this stage, *Process1*'s label is: $[(Process, Process, Socket, Clone, Connect), (Process, File, Process, Clone, Write), (Process, Process, Clone), (Socket, Process, Connect), Clone, Connect)]$.

Appendix C. MIRDETECTOR's analysis for [Fig. 1](#)

In line with the detection methodology of MIRDETECTOR, we revisit the analysis of [Fig. 1\(b\)](#), (c), and (d). For [Fig. 1\(b\)](#), there is a marked alteration in its structural characteristics, leading to an increased loss in its structural feature vector; MIRDETECTOR is capable of detecting such attacks. In the case of [Fig. 1\(c\)](#), while the structural characteristics do not vary drastically, there is a noticeable change in its attribute features, and the nodes display evident malicious intent (for instance: the *vUgefal* node executes a file sourced externally and reads the */etc/passwd* file, then establishes an external connection. As per reports and whitepapers ([Al-Mhiqani et al., 2020](#)), this aligns with attack characteristics: the acquisition of sensitive information, hence the losses in its attribute and MIR features will escalate, and MIRDETECTOR can identify such attacks. Regarding [Fig. 1\(d\)](#), even though there is a change in structural characteristics, there is no clear indication of malicious intent features, resulting in a relatively minor increase in MIR feature loss, with the overall loss not surpassing the established threshold; thus, MIRDETECTOR will classify it as benign behavior. Consequently, by conducting sound modeling and judiciously assigning weights to structural features, attribute features, and MIR features, it is feasible to diminish false positives, false negatives, and bolster detection efficacy.

Data availability

Data will be made available on request.

Table 12
Definition of process labels.

Label	Description	Category
PF1	The process has a network connection	Feature
PF2	The process contains data downloaded from the network	Feature
PF3	The process has opened files downloaded from the network	Feature
PF4	The process has read files containing user information	Feature
PF5	The process has read files related to log information	Feature
PF6	The process interacts with non-existent files	Feature
PB1	The process executes files obtained from the network	Behavior
PB2	The process executes sensitive files	Behavior
PB3	The process executes sensitive commands	Behavior
PB4	The process executes shell commands	Behavior
PB5	The process modifies cron files	Behavior
PB6	The process reads confidential data	Behavior
PB7	The process modifies permission files	Behavior
PB8	The process sends data over a network connection	Behavior

Table 13
Definition of file labels.

Label	Description	Category
FU1	The file contains network data	Untrusted
FU2	The file does not exist	Untrusted
FU3	The file is created through network upload	Untrusted
FC1	Files that control user permissions such as /etc/sudoers	Confidential
FC2	Files that store sensitive information such as /etc/passwd	Confidential
FC3	Files that control scheduled tasks such as like /etc/crontab	Confidential
FC4	Files that save the history of executed commands such as .bash_history	Confidential
FC5	Files created by processes that have read confidential data	Confidential

References

2024. Adversarial tactics, techniques, and common knowledge, <https://attack.mitre.org/wiki/MainPage>.
- Al-Mhiqani, Mohammed, Ahmad, Rabiah, Zainal Abidin, Zaheera, Mohamed, Warusia, Hassan, Aslinda, Abdulkareem, Karrar, Ali, Nabeel, Yunos, Zahri, 2020. A review of insider threat detection: Classification, machine learning techniques, datasets, open challenges, and recommendations. *Appl. Sci.* 10, 1–41. <http://dx.doi.org/10.3390/app10155208>.
- Alsaheel, Abdulallah, Nan, Yuhong, Ma, Shiqing, Yu, Le, Walkup, Gregory, Celik, Z Berkay, Zhang, Xiangyu, Xu, Dongyan, 2021. {Atlas}: A sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 3005–3022.
- Alshamrani, Adel, Myneni, Sowmya, Chowdhary, Ankur, Huang, Dijiang, 2019. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. *IEEE Commun. Surv. Tutor.* 21 (2), 1851–1877.
- Bates, Adam, Tian, Dave Jing, Butler, Kevin RB, Moyer, Thomas, 2015. Trustworthy {whole-system} provenance for the linux kernel. In: 24th USENIX Security Symposium. USENIX Security 15, pp. 319–334.
- Chen, Zhuangbin, Liu, Jinyang, Gu, Wenwei, Su, Yuxin, Lyu, Michael R, 2021. Experience report: Deep learning-based system log analysis for anomaly detection. *arXiv preprint arXiv:2107.05908*.
- Cheng, Zijun, Lv, Qiujian, Liang, Jinyuan, Wang, Yan, Sun, Degang, Pasquier, Thomas, Han, Xueyuan, 2023. Kairos: Practical intrusion detection and investigation using whole-system provenance. *ArXiv, arXiv:2308.05034*.
- Chum, Ondrej, Philbin, James, Zisserman, Andrew, et al., 2008. Near duplicate image detection: Min-hash and TF-IDF weighting. In: *Bmvc*. Vol. 810, pp. 812–815.
2020. DARPA transparent computing engagement. <https://www.darpa.mil/program/transparent-computing>.
- Fang, Pengcheng, Gao, Peng, Liu, Changlin, Ayday, Erman, Jee, Kangkook, Wang, Ting, Ye, Yanfang Fanny, Liu, Zhuotao, Xiao, Xusheng, 2021. Back-propagating system dependency impact for attack investigation. In: 31st USENIX Security Symposium, Boston, MA. USENIX Security, pp. 10–12.
2024. GENSIM: Topic modelling for humans. <https://radimrehurek.com/gensim/>.
- Goyal, Akul, Han, Xueyuan, Wang, Gang, Bates, Adam, 2023. Sometimes, you aren't what you do: Mimicry attacks against provenance graph host intrusion detection systems. In: 30th Network and Distributed System Security Symposium.
- Hamilton, Will, Ying, Zhitaio, Leskovec, Jure, 2017. Inductive representation learning on large graphs. *Adv. Neural Inf. Process. Syst.* 30.
- Han, Xueyuan, Pasquier, Thomas, Bates, Adam, Mickens, James, Seltzer, Margo, 2020. Unicorn: Runtime provenance-based detector for advanced persistent threats. *arXiv preprint arXiv:2001.01525*.
- Han, Xueyuan, Pasquier, Thomas, Ranjan, Tanvi, Goldstein, Mark, Seltzer, Margo, 2017. FRAPPuccino: Fault-detection through runtime analysis of provenance. In: *Proceedings of the 9th USENIX Conference on Hot Topics in Cloud Computing*. p. 18.
- Han, Xueyuan, Yu, Xiao, Pasquier, Thomas, Li, Ding, Rhee, Junghwan, Mickens, James, Seltzer, Margo, Chen, Haifeng, 2021. SIGL: Securing software installations through deep graph learning. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 2345–2362, URL <https://www.usenix.org/conference/usenixsecurity21/presentation/han-xueyuan>.
- Hasan, Ragib, Sion, Radu, Winslett, Marianne, 2009. Preventing history forgery with secure provenance. *ACM Trans. Storage* 5, 12:1–12:43.
- Hassan, Wajih Ul, Guo, Shengjian, Li, Ding, Chen, Zhengzhang, Jee, Kangkook, Li, Zhichun, Bates, Adam, 2019a. Nododo: Combatting threat alert fatigue with automated provenance triage. In: *Network and Distributed Systems Security Symposium*.
- Hassan, Wajih Ul, Guo, Shengjian, Li, Ding, Chen, Zhengzhang, Jee, Kangkook, Li, Zhichun, Bates, Adam, 2019b. NoDoze: Combatting threat alert fatigue with automated provenance triage. *Proc. 2019 Netw. Distrib. Syst. Secur. Symp.*.
- Hassan, Wajih, Lemay, Mark, Aguse, Nuraini, Bates, Adam, Moyer, Thomas, 2018. Towards scalable cluster auditing through grammatical inference over provenance graphs. In: *Network and Distributed Systems Security Symposium*.
- He, Shilin, Zhu, Jieming, He, Pinjia, Lyu, Michael R, 2016. Experience report: System log analysis for anomaly detection. In: 2016 IEEE 27th International Symposium on Software Reliability Engineering. ISSRE, IEEE, pp. 207–218.
- Hossain, Md Nahid, Milajerdi, Sadeh M, Wang, Junao, Eshete, Birhanu, Gjomemo, Rigel, Sekar, R, Stoller, Scott, Venkatakrishnan, VN, 2017a. {SLEUTH}: Real-time attack scenario reconstruction from COTS audit data. In: *Proceedings of the 26th USENIX Conference on Security Symposium*. pp. 487–504.
- Hossain, Md Nahid, Milajerdi, Sadeh M, Wang, Junao, Eshete, Birhanu, Gjomemo, Rigel, Sekar, R, Stoller, Scott, Venkatakrishnan, VN, 2017b. {SLEUTH}: Real-time attack scenario reconstruction from {COTS} audit data. In: *USENIX Security Symposium*. pp. 487–504.
- Hwang, Frank K., Richards, Dana S., 1992. Steiner tree problems. *Netw.* 22 (1), 55–89.
- Imase, Makoto, Waxman, Bernard M., 1991. Dynamic steiner tree problem. *SIAM J. Discrete Math.* 4 (3), 369–384.
- Inam, Muhammad Adil, Chen, Yinfang, Goyal, Akul, Liu, Jason, Mink, Jaron, Michael, Noor, Gaur, Sneha, Bates, Adam, Hassan, Wajih Ul, 2022. SoK: History is a vast early warning system: Auditing the provenance of system intrusions. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE Computer Society, pp. 307–325.
- Jia, Zian, Xiong, Yun, Nan, Yuhong, Zhang, Yao, Zhao, Jinjing, Wen, Mi, 2024. {MAGIC}: Detecting advanced persistent threats via masked graph representation learning. In: 33rd USENIX Security Symposium. USENIX Security 24, pp. 5197–5214.
- Joulin, Armand, Grave, Edouard, Bojanowski, Piotr, Mikolov, Tomas, 2016. Bag of tricks for efficient text classification. *arXiv preprint arXiv:1607.01759*.
- Kaufman, Leonard, Rousseeuw, Peter, 2009. *Finding Groups in Data: An Introduction to Cluster Analysis*. John Wiley & Sons.
- Keromytis, A.D., 2018. Transparent computing engagement 3 data release. <https://github.com/darpa-i2o/TransparentComputing/blob/master/README-E3.md>.

- Lai, Siwei, Xu, Liheng, Liu, Kang, Zhao, Jun, 2015. Recurrent convolutional neural networks for text classification. In: AAAI Conference on Artificial Intelligence.
- Le, Quoc, Mikolov, Tomas, 2014. Distributed representations of sentences and documents. In: International Conference on Machine Learning. PMLR, pp. 1188–1196.
- Liu, Yushan, Zhang, Mu, Li, Ding, Jee, Kangkook, Li, Zhichun, Wu, Zhenyu, Rhee, Junghwan, Mittal, Prateek, 2018. Towards a timely causality analysis for enterprise security. In: NDSS.
2022. Lockheed martin. Cyber kill chain. <https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html>.
- Lv, Mingqi, Gao, Hongzhe, Qiu, Xuebo, Chen, Tieming, Zhu, Tiantian, Chen, Jinyin, Ji, Shouling, 2024. TREC: APT tactic/technique recognition via few-shot provenance subgraph learning. In: Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security. pp. 139–152.
- Manzoor, 2016. Sbustreamspot-data. <https://github.com/sbustreamspot/sbustreamspot-data>.
- Manzoor, Emaad, Milajerdi, Sadegh M., Akoglu, Leman, 2016. Fast memory-efficient anomaly detection in streaming heterogeneous graphs. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1035–1044.
- Mikolov, Tomas, 2013. Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.
- Milajerdi, Sadegh M, Eshete, Birhanu, Gjomemo, Rigel, Venkatakrishnan, VN, 2019a. Poirrot: Aligning attack behavior with kernel audit records for cyber threat hunting. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 1795–1812.
- Milajerdi, Sadegh M, Gjomemo, Rigel, Eshete, Birhanu, Sekar, Ramachandran, Venkatakrishnan, VN, 2019b. HOLMES: real-time APT detection through correlation of suspicious information flows. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1137–1152.
- Murfnová, Júlia, 2015. Application log analysis. Relat. Técnico, Masaryk. Univ. Fak. Inform..
- Najafi, Pejman, Puente, Wenzel, Cheng, Feng, Meinel, Christoph, 2024. You are your friends: Detecting malware via guilt-by-association and exempt-by-reputation. Comput. Secur. 136, 103519.
- Narayanan, Annamalai, Chandramohan, Mahinthan, Venkatesan, Rajasekar, Chen, Lihui, Liu, Yang, Jaiswal, Shantanu, 2017. Graph2vec: Learning distributed representations of graphs. arXiv preprint arXiv:1707.05005.
- Navarro, Julio, Deruyver, Aline, Parrend, Pierre, 2018. A systematic survey on multi-step attack detection. Comput. Secur. 76, 214–249.
2024. NetworkX: Network analysis in python. <https://networkx.org/>.
- Paccagnella, Riccardo, Datta, Pubali, Hassan, Wajih Ul, Bates, Adam, Fletcher, Christopher, Miller, Andrew, Tian, Dave, 2020a. Custos: Practical tamper-evident auditing of operating systems using trusted execution. In: Network and Distributed System Security Symposium.
- Paccagnella, Riccardo, Liao, Kevin, Tian, Dave, Bates, Adam, 2020b. Logging to the danger zone: Race condition attacks and defenses on system audit frameworks. In: Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. pp. 1551–1574.
- Pang, Guansong, Shen, Chunhua, Cao, Longbing, Hengel, Anton Van Den, 2021. Deep learning for anomaly detection: A review. ACM Comput. Surv. (CSUR) 54 (2), 1–38.
- Pasquier, Thomas, Han, Xueyuan, Goldstein, Mark, Moyer, Thomas, Eysers, David, Seltzer, Margo, Bacon, Jean, 2017. Practical whole-system provenance capture. In: Proceedings of the 2017 Symposium on Cloud Computing. pp. 405–418.
- Pohly, Devin J, McLaughlin, Stephen, McDaniel, Patrick, Butler, Kevin, 2012. Hi-fi: collecting high-fidelity whole-system provenance. In: Proceedings of the 28th Annual Computer Security Applications Conference. pp. 259–268.
2024. Pytorch geometric. <https://pytorch-geometric.readthedocs.io/en/latest/index.html>.
- Rehman, Mati Ur, Ahmadi, Hadi, Hassan, Wajih Ul, 2024. Flash: A comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 3552–3570.
- Rossi, Emanuele, Chamberlain, Ben, Frasca, Fabrizio, Eynard, Davide, Monti, Federico, Bronstein, Michael, 2006. Temporal graph networks for deep learning on dynamic graphs. arXiv 2020, arXiv preprint arXiv:2006.10637.
- Rossi, Emanuele, Chamberlain, Ben, Frasca, Fabrizio, Eynard, Davide, Monti, Federico, Bronstein, Michael, 2020. Temporal graph networks for deep learning on dynamic graphs. arXiv preprint arXiv:2006.10637.
- Shervashidze, Nino, Schweitzer, Pascal, Van Leeuwen, Erik Jan, Mehlhorn, Kurt, Borgwardt, Karsten M, 2011. Weisfeiler–Lehman graph kernels. J. Mach. Learn. Res. 12 (9).
- Torrey, J., 2020. Transparent computing engagement 5 data release. <https://github.com/darpa-i2o/Transparent-Computing>.
- Wang, Binghui, Gong, Neil Zhenqiang, 2019. Attacking graph-based classification via manipulating the graph structure. In: Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security. pp. 2023–2040.
- Wang, Qi, Hassan, Wajih Ul, Li, Ding, Jee, Kangkook, Yu, Xiao, Zou, Kexuan, Rhee, Junghwan, Chen, Zhengzhang, Cheng, Wei, Gunter, Carl A, et al., 2020. You are what you do: Hunting stealthy malware via data provenance analysis. In: NDSS.
- Wang, Su, Wang, Zhiqiang, Zhou, Tao, Sun, Hongbin, Yin, Xia, Han, Dongqi, Zhang, Han, Shi, Xingang, Yang, Jiahai, 2022. THREATTRACE: Detecting and tracing host-based threats in node level through provenance graph learning. IEEE Trans. Inf. Forensics Secur. 3972–3987.
- Xie, Yulai, Feng, Dan, Hu, Yuchong, Li, Yan, Sample, Staunton, Long, Darrell, 2018. Pagoda: A hybrid approach to enable efficient real-time provenance based intrusion detection in big data environments. IEEE Trans. Dependable Secur. Comput. 17 (6), 1283–1296.
- Xu, Jiachen, Stokes, Jack W, McDonald, Geoff, Bai, Xuesong, Marshall, David, Wang, Siyue, Swaminathan, Adith, Li, Zhou, 2024. Autoattacker: A large language model guided system to implement automatic cyber-attacks. arXiv preprint arXiv:2403.01038.
- Yang, Dingqi, Li, Bin, Rettig, Laura, Cudré-Mauroux, Philippe, 2017. Histosketch: Fast similarity-preserving sketching of streaming histograms with concept drift. In: 2017 IEEE International Conference on Data Mining. ICDM, IEEE, pp. 545–554.
- Yang, Fan, Xu, Jiachen, Xiong, Chunlin, Li, Zhou, Zhang, Kehuan, 2023. PROGRAPHER: An anomaly detection system based on provenance graph embedding. In: Proceedings of the 32nd USENIX Conference on Security Symposium.
- Ying, Jie, Zhu, Tiantian, Cheng, Wenrui, Yuan, Qixuan, Ma, Mingjun, Xiong, Chunlin, Chen, Tieming, Lv, Mingqi, Chen, Yan, 2024. SPARSE: Semantic tracking and path analysis for attack investigation in real-time. arXiv preprint arXiv:2405.02629.
- Zengy, Jun, Wang, Xiang, Liu, Jiahao, Chen, Yinfang, Liang, Zhenkai, Chua, Tat-Seng, Chua, Zheng Leong, 2022. SHADEWATCHER: Recommendation-guided cyber threat analysis using system audit records. In: 2022 IEEE Symposium on Security and Privacy. SP, pp. 489–506.
- Zhang, Zhaoqi, Qi, Panpan, Wang, Wei, 2020. Dynamic malware analysis with feature engineering and feature learning. In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 34, pp. 1210–1217.
- Zhu, Tiantian, Yu, Jinkai, Xiong, Chunlin, Cheng, Wenrui, Yuan, Qixuan, Ying, Jie, Chen, Tieming, Zhang, Jiabo, Lv, Mingqi, Chen, Yan, Wang, Ting, Fan, Yuan, 2023. APTSHIELD: A stable, efficient and real-time APT detection system for linux hosts. IEEE Trans. Dependable Secur. Comput..
- Zügner, Daniel, Akbarnejad, Amir, Günnemann, Stephan, 2018. Adversarial attacks on neural networks for graph data. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. pp. 2847–2856.

Hongmei Li received the B.E. degree in computer science from the Henan University of Traditional Chinese Medicine, Zhengzhou, China in 2023. Her is currently pursuing the M.S. degree for computer science in Zhejiang University of Technology. Her current research interests and directions include cyber threat detection.

Tiantian Zhu received the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2019. He is currently an associate professor with the college of computer science and technology, Zhejiang University of Technology, China. His research interests include mobile security, system security and artificial intelligence.

Jie Ying received the B.E. degree in digital media from Zhejiang University of Technology, Hangzhou, China, in 2020. He is currently pursuing Ph.D. degree for computer science in Zhejiang University of Technology. His current research interests include mobile security and Cyber Threat Intelligence.

Tieming Chen received the Ph.D. degree in computer software and theory from BeiHang University, Beijing, China, in 2011. He is now a professor with the college of computer science and technology, Zhejiang University of Technology, Hangzhou, China. He is also a member of IEEE and ACM. His research interests include cyberspace security and intelligence security.

Mingqi Lv received the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2012. His research interests include spatiotemporal data mining and ubiquitous computing.

Jian-Ping Mei received the Ph.D. degree in information engineering from Nanyang Technological University, Singapore, in 2012. Her research interests include Machine learning and data mining.

Zhengqiu Weng received the Ph.D. degree in computer science and technology from the Zhejiang University of Technology, Zhejiang, China, in 2023, and the M.S. degree in software engineering from the Beijing Institute of Technology, Beijing, China, in 2005. She is also a professor with the Wenzhou University of Technology, Zhejiang, China. Her current research interests include mobile security and big data technologies.

Lili Shi received the M.S. degree from Zhejiang Normal University in 2014. She is also a lecturer in Wenzhou Polytechnic. Her research interests include data analysis and visualization, data security.